

Value function iterations

Contraction mapping theorem provides a very simple method for computing the value function numerically (when the closed-form solution does not exist)

Guess TV

```
while V ≠ TV          iterate on V till V = TV
  V = TV
  compute TV
end
```

To implement it:

tol = 10^{-6} (small)

set the tolerance level to "Judge" if V is suff. close to TV

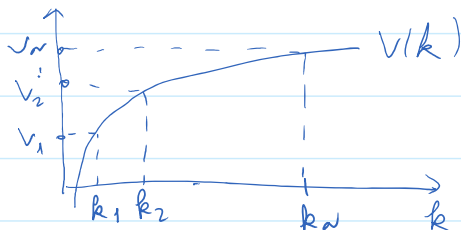
TV = (guess, e.g. 0)

Diff = 1

```
while Diff > tol
  V = TV
  TV = max_{k'} { F(k, k') + β V(k') }
  Diff = ||V - TV||
end
```

More details:

• How to define $V(k)$ (no analytical expression)?



Create a grid:

$[k_1 \dots k_n]$

Then a function $V(k)$ is described by $[V_1 \dots V_n]$ (values on this grid)

In practice,

$k_{\text{grid}} = \text{linspace}(k_{\text{min}}, k_{\text{max}}, N)$

uniformly spaced
between k_{min} & k_{max}

Then the goal of the code is to find $[V_1 \dots V_n]$ such that

$$[V_1 \dots V_N] = [TV_1 \dots TV_N]$$

$\Rightarrow$ set $k_{\min}, k_{\max}, N$, define k_{grid} in the beginning of the code.

• How to compute $|TV - V|$ - ?

utility values are not meaningful! $\Rightarrow$

$\Rightarrow$ use "consumption equivalence"!

Let c_v be cons. level:

$$V = \frac{\sum_{t=0}^{\infty} \beta^t u(c_t)}{1-\beta} = \frac{u(c_v)}{1-\beta}$$

$\Rightarrow$ measure $|TV - V| = |c_v - c_{TV}|$, i.e.

$$\max | \text{abs} (u^{-1}((1-\beta)V) - u^{-1}((1-\beta)TV)) |$$

• How to find $\max_{k' \leq t(k)} \{ F(k, k') + V(k') \}$

Most obvious:

```

for i = 1:N
    [TV(i) indg(i)] = max ( F(kgrid(i), kgrid) +  $\beta$  V )
    ↑
    index of the element that delivers the highest value
end

```

number $\swarrow$ vector $[k_1 \dots k_N]$
vector $[V_1 \dots V_N]$

Loops are slow. Can do the same in one row using matrices:

$$\max \begin{pmatrix} F(k_1, k_1) + \beta V_1 & F(k_2, k_1) + \beta V_1 & \dots & F(k_N, k_1) + \beta V_1 \\ F(k_1, k_2) + \beta V_2 & F(k_2, k_2) + \beta V_2 & \dots & F(k_N, k_2) + \beta V_2 \\ \vdots & \vdots & \ddots & \vdots \\ F(k_1, k_N) + \beta V_N & F(k_2, k_N) + \beta V_N & \dots & F(k_N, k_N) + \beta V_N \end{pmatrix}$$

$(N \times N)$ matrix

would find the max in every column

to build such a matrix:

$$kk = \text{kron}(kgrid, \text{ones}(N, 1))$$

$$[TV \text{ indg}] = \max (F(kk, k') + \beta \text{kron}(V', \text{ones}(1, N)))$$

this line replaces the whole loop & computes TV for a given V

Finally, notice that $F(kk, k')$ depends on the grid only $\Rightarrow$ no need to recompute it inside the while loop for any new guess of V.

$\Rightarrow$ structure the code as follows:

set tolerance tol

define kgrid

find kk, $util = F(kk, k')$

guess TV

set diff = 1 (any > tol)

while diff > tol

$V = TV$

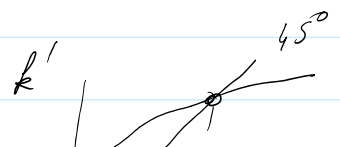
$[TV \text{ indg}] = \max \dots$

$\text{diff} = \max (abs(u'(1-\beta)V) - u'(1-\beta)TV))$

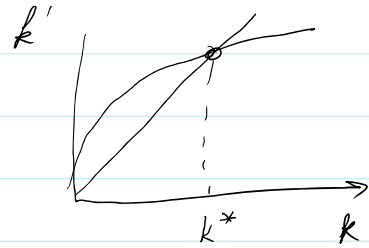
end.

Remark: $kgrid(\text{indg})$ is the policy function!
(e.g. $\text{plot}(kgrid, kgrid(\text{indg}))$)
would plot $k' = g(k)$

Some things to worry about:



Some things to worry about:



- ① $k_{\max} (k_w)$ must be large enough
($k_{\max} > k^*$)
check that $k_{\max}$ is not binding ($g(k) < k_{\max}, \forall k$)
- ② N - large enough
increase $N \Rightarrow V, g$ must not be significantly affected
- ③ tol must be small enough
decrease tol , make sure that V, g are not much affected

Numerical solution - how to speed things up

Sometimes the brute-force value function iteration is too slow (multiple state variables - "curse of dimensionality")

$\Rightarrow \exists$ tricks to speed up the algorithm:

- 1) Evaluate V on a sparser grid of $[k_1, \dots, k_m]$.
Interpolate V on the intermediate levels of k
(use `interp1` in Matlab)
Choose optimal k' from a denser grid.

The grid for k may be chosen unevenly:
more grid points in the region in which V is more concave (small k)

- 2) Use the properties of $V(k)$, $g(k)$ in your computations³
(if you can establish analytical properties)

Example: $g(k)$ - monotone $\Rightarrow$
the set of k' over which you need to search adjusts with k
(fewer computations will be made $\Rightarrow$ faster)

- 3) Howard's Improvement algorithm:

(a) compute $g(k)$ for a given V -guess

(b) compute $V(k)$ induced by $g(k)$

$$V(k) = F(k, g(k)) + \beta V(g(k)) \quad (\text{no max!})$$

Or apply the RHS of the above 10-20 times, not till converge to the solution
(max is time consuming)

• repeat (a)-(b) till $g(k)$ does not change.

(sometimes does not converge)

4) Solve numerically the Euler Equation

5) Continuous time formulation -- easier to solve (see work of Ben Moll)

6) neural networks methods to approximate value function or policy functions (using eg Deep Learning Toolbox in Matlab)